

Lecture 14: BFS/DFS for Routing and Reachability

Topics

- Breadth-First Search (BFS) with `collections.deque`
- Depth-First Search (DFS) with stack and recursion
- Finding shortest paths with BFS
- Discovering connected components
- Detecting cycles in graphs
- Applications: artist collaboration paths, music genre clustering

Goals

- Implement BFS for shortest path finding
- Implement DFS with both iterative and recursive approaches
- Find connected components in collaboration networks
- Detect cycles to find circular dependencies
- Compute distances and paths in real graphs

Roadmap

First half (≈ 35 min)

- BFS fundamentals: level-by-level exploration
- Implementing BFS with deque
- Finding shortest paths with BFS
- Path reconstruction from parent pointers
- Distance computation
- In-class exercise 1 (commit required)

Break (3 min)

Second half (≈ 35 min)

- DFS fundamentals: deep exploration
- Iterative DFS with stack
- Recursive DFS implementation
- Connected components discovery
- Cycle detection in graphs
- In-class exercise 2 (commit required)
- Complexity table and wrap-up

Setup: load data and build graph

```
In [ ]: import csv
from collections import deque, defaultdict
from typing import Dict, Set, List, Optional, Tuple

# Load tracks
tracks = {}
with open('data/tracks.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        track_id = row['track_id'].strip().lower()
        tracks[track_id] = row

# Load artists
artists = {}
with open('data/artists.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        artists[row['artist_id']] = row['artist_name']

# Join artist names into tracks
for track in tracks.values():
    artist_id = track.get('artist_id', '')
    track['artist'] = artists.get(artist_id, f"Unknown ({artist_id})")

# Build artist collaboration graph
def build_artist_graph(tracks: dict) -> Dict[str, Set[str]]:
```

```

"""Build graph of artists who share similar characteristics."""
graph = defaultdict(set)
artists_list = list(set(track['artist'] for track in tracks.values()))

# For demo: connect artists with similar names or from same tracks
# In reality, you'd use collaboration data, genre, etc.
for i, artist1 in enumerate(artists_list[:30]):
    for artist2 in artists_list[i+1:i+4]: # Connect to next few artists
        if artist2 != artist1:
            graph[artist1].add(artist2)
            graph[artist2].add(artist1)

return dict(graph)

artist_graph = build_artist_graph(tracks)
print(f"Built graph with {len(artist_graph)} artists")
print(f"Sample artist: {list(artist_graph.keys())[0]}")
print(f"    has {len(list(artist_graph.values())[0])} connections")

```

Part 1: Breadth-First Search (BFS)

BFS explores a graph level by level, like ripples in water.

Key idea: Visit all neighbors at distance 1, then all at distance 2, then distance 3, etc.

Why BFS?

- Finds shortest path in unweighted graphs
- Explores nearby nodes first
- Level-order traversal

Data structure: Queue (FIFO) using `collections.deque`

```
Start: A
Level 0: [A]
Level 1: [B, C] (neighbors of A)
Level 2: [D, E, F] (neighbors of B and C)
Level 3: [G] (neighbors of D, E, F)
```

BFS algorithm

Algorithm:

1. Create queue, add start vertex
2. Mark start as visited
3. While queue not empty:
 - Dequeue vertex
 - For each unvisited neighbor:
 - Mark as visited
 - Enqueue neighbor

Key: Use queue (FIFO) to process nodes in order discovered.

```
In [ ]: def bfs(graph: Dict[str, Set[str]], start: str) -> Set[str]:  
        """BFS traversal from start vertex."""  
        visited = set()  
        queue = deque([start])  
        visited.add(start)  
  
        while queue:  
            # Dequeue from front (FIFO)  
            current = queue.popleft()  
            print(f"Visiting: {current}")
```

```
    # Explore neighbors
    for neighbor in graph.get(current, set()):
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)

    return visited

# Test BFS
if artist_graph:
    start_artist = list(artist_graph.keys())[0]
    print(f"\nBFS from {start_artist}:\n")
    reachable = bfs(artist_graph, start_artist)
    print(f"\nReached {len(reachable)} artists from {start_artist}")
```

BFS for shortest path

Problem: Find shortest path between two artists in collaboration network.

Solution: Track parent pointers during BFS, reconstruct path afterward.

Why BFS finds shortest path:

- BFS visits nodes in order of increasing distance
- First time we reach target = shortest path
- Guaranteed for unweighted graphs

```
In [ ]: def bfs_shortest_path(graph: Dict[str, Set[str]],
                                start: str,
                                target: str) -> Optional[List[str]]:
    """Find shortest path from start to target using BFS."""
    if start not in graph or target not in graph:
        return None

    if start == target:
        return [start]

    visited = set()
    queue = deque([start])
    visited.add(start)
    parent = {start: None}  # Track parent for path reconstruction
```

```

while queue:
    current = queue.popleft()

    # Check if we found target
    if current == target:
        # Reconstruct path
        path = []
        node = target
        while node is not None:
            path.append(node)
            node = parent[node]
        return path[::-1] # Reverse to get start->target

    # Explore neighbors
    for neighbor in graph.get(current, set()):
        if neighbor not in visited:
            visited.add(neighbor)
            parent[neighbor] = current
            queue.append(neighbor)

return None # No path found

# Test shortest path
if len(artist_graph) >= 2:
    artists = list(artist_graph.keys())
    start = artists[0]
    target = artists[min(5, len(artists)-1)]

    path = bfs_shortest_path(artist_graph, start, target)
    if path:
        print(f"\nShortest path from {start} to {target}:")

```

```
    for i, artist in enumerate(path):  
        print(f" {i}: {artist}")  
    print(f"\nPath length: {len(path) - 1} steps")  
else:  
    print(f"No path from {start} to {target}")
```

BFS with distances

Compute distance (number of edges) from start to all reachable vertices.

```
In [ ]: def bfs_distances(graph: Dict[str, Set[str]], start: str) -> Dict[str, int]:
        """Compute distance from start to all reachable vertices."""
        distances = {start: 0}
        queue = deque([start])

        while queue:
            current = queue.popleft()
            current_dist = distances[current]

            for neighbor in graph.get(current, set()):
                if neighbor not in distances:
                    distances[neighbor] = current_dist + 1
                    queue.append(neighbor)

        return distances

# Test distances
if artist_graph:
    start = list(artist_graph.keys())[0]
    distances = bfs_distances(artist_graph, start)

    print(f"\nDistances from {start}:\n")
    # Group by distance
    by_distance = defaultdict(list)
```

```
for artist, dist in distances.items():
    by_distance[dist].append(artist)

for dist in sorted(by_distance.keys()):
    print(f"Distance {dist}: {len(by_distance[dist])} artists")
    if dist <= 2: # Show first few levels
        for artist in by_distance[dist][:3]:
            print(f"    {artist}")
```

BFS applications

Real-world uses:

- **Shortest path:** Google Maps (unweighted roads)
- **Social networks:** Degrees of separation ("6 degrees of Kevin Bacon")
- **Web crawling:** Explore web pages level by level
- **Puzzle solving:** Find minimum moves to solution
- **Network broadcasting:** Spread message to all nodes

Spotify applications:

- Find collaboration path between two artists
- Discover artists within k degrees of separation
- Recommend artists based on network proximity
- Analyze music genre neighborhoods

Complexity:

- Time: $O(V + E)$ — visit each vertex and edge once
- Space: $O(V)$ — store visited set and queue

Exercise 1: Find collaboration chains

Task: Find the shortest collaboration chain between two artists.

Requirements:

1. Build a graph where artists are connected if they share a track
2. Implement `find_collaboration_chain(graph, artist1, artist2)`
3. Use BFS to find shortest path
4. Print the chain: "Artist A → Artist B → Artist C"
5. Print the chain length (degrees of separation)

Hint: Use `bfs_shortest_path` as a starting point.

Commit required: Commit your solution before the break.

```
In [ ]: # YOUR CODE HERE
        # Find collaboration chain between artists

def find_collaboration_chain(graph, artist1, artist2):
    """Find shortest collaboration chain between two artists."""
    # TODO: Use BFS to find shortest path
    # TODO: Return list of artists in chain
    pass
```

```
# Test with two artists from your graph  
# chain = find_collaboration_chain(artist_graph, artist1, artist2)
```

Break (3 minutes)

Commit your Exercise 1 solution now!

When we return:

- Depth-First Search (DFS)
- Connected components
- Cycle detection

Part 2: Depth-First Search (DFS)

DFS explores as deeply as possible before backtracking.

Key idea: Follow one path until you can't go further, then backtrack.

Why DFS?

- Detect cycles
- Find connected components
- Topological sorting
- Maze solving

Data structure: Stack (LIFO) or recursion

```
Start: A
Visit: A → B → D → G (dead end, backtrack)
       A → B → E (dead end, backtrack)
       A → C → F (dead end, backtrack)
```

Iterative DFS with stack

```
In [ ]: def dfs_iterative(graph: Dict[str, Set[str]], start: str) -> Set[str]:
        """DFS traversal using explicit stack."""
        visited = set()
        stack = [start]  # Use list as stack

        while stack:
            # Pop from back (LIFO)
            current = stack.pop()

            if current not in visited:
                visited.add(current)
                print(f"Visiting: {current}")

                # Add neighbors to stack
                for neighbor in graph.get(current, set()):
                    if neighbor not in visited:
                        stack.append(neighbor)

        return visited

# Test DFS
if artist_graph:
    start_artist = list(artist_graph.keys())[0]
    print(f"\nDFS from {start_artist}:\n")
    reachable = dfs_iterative(artist_graph, start_artist)
    print(f"\nReached {len(reachable)} artists from {start_artist}")
```

Recursive DFS

More elegant: Use call stack instead of explicit stack.

```
In [ ]: def dfs_recursive(graph: Dict[str, Set[str]],
                        vertex: str,
                        visited: Set[str]) -> None:
    """DFS traversal using recursion."""
    visited.add(vertex)
    print(f"Visiting: {vertex}")

    for neighbor in graph.get(vertex, set()):
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)

def dfs(graph: Dict[str, Set[str]], start: str) -> Set[str]:
    """Wrapper for recursive DFS."""
    visited = set()
    dfs_recursive(graph, start, visited)
    return visited

# Test recursive DFS
if artist_graph:
    start_artist = list(artist_graph.keys())[0]
    print(f"\nRecursive DFS from {start_artist}:\n")
    reachable = dfs(artist_graph, start_artist)
    print(f"\nReached {len(reachable)} artists")
```

BFS vs DFS comparison

Property	BFS	DFS
Data structure	Queue (deque)	Stack (list or recursion)
Order	Level by level	Deep then backtrack
Shortest path	✅ Yes (unweighted)	❌ No
Space	$O(V)$ worst case	$O(\text{depth})$ average
Time	$O(V + E)$	$O(V + E)$
Use for	Shortest paths, level order	Cycles, components, topological

When to use:

- **BFS:** Finding shortest paths, exploring nearby nodes
- **DFS:** Detecting cycles, finding components, backtracking problems

Connected components

Problem: Find all separate groups (components) in a graph.

Example: Find clusters of artists who are all connected via collaborations.

Algorithm:

1. Start with all vertices unvisited
2. For each unvisited vertex:
 - Run DFS/BFS to find its component
 - Mark all reached vertices as visited
3. Each DFS/BFS run finds one component

```
In [ ]: def find_connected_components(graph: Dict[str, Set[str]]) -> List[Set[str]]:
        """Find all connected components in graph."""
        visited = set()
        components = []

        for vertex in graph:
            if vertex not in visited:
                # Find component containing this vertex
                component = set()
                stack = [vertex]

                while stack:
```

```

        current = stack.pop()
        if current not in visited:
            visited.add(current)
            component.add(current)

            for neighbor in graph.get(current, set()):
                if neighbor not in visited:
                    stack.append(neighbor)

    components.append(component)

    return components

# Find components
components = find_connected_components(artist_graph)
print(f"\nFound {len(components)} connected components:\n")
for i, component in enumerate(components):
    print(f"Component {i+1}: {len(component)} artists")
    if len(component) <= 5:
        print(f"  Artists: {component}")

```

Cycle detection

Problem: Does the graph contain a cycle?

Cycle: Path that starts and ends at same vertex.

Algorithm (undirected graph):

- Use DFS
- If we visit a vertex that's already visited AND it's not our parent, we found a cycle

```
In [ ]: def has_cycle_undirected(graph: Dict[str, Set[str]]) -> bool:
        """Check if undirected graph has a cycle."""
        visited = set()

        def dfs_cycle(vertex: str, parent: Optional[str]) -> bool:
            """DFS that detects cycles."""
            visited.add(vertex)

            for neighbor in graph.get(vertex, set()):
                if neighbor not in visited:
                    # Recurse on unvisited neighbor
                    if dfs_cycle(neighbor, vertex):
                        return True
                elif neighbor != parent:
                    # Found visited vertex that's not our parent = cycle!
                    return True
```

```
    return False
```

```
    # Check each component
```

```
    for vertex in graph:
```

```
        if vertex not in visited:
```

```
            if dfs_cycle(vertex, None):
```

```
                return True
```

```
    return False
```

```
# Check for cycles
```

```
has_cycle = has_cycle_undirected(artist_graph)
```

```
print(f"\nArtist collaboration graph has cycles: {has_cycle}")
```

```
print("(Expected: True for most connected graphs)")
```

Finding a cycle (if exists)

```
In [ ]: def find_cycle(graph: Dict[str, Set[str]]) -> Optional[List[str]]:
        """Find a cycle in the graph (if one exists)."""
        visited = set()

        def dfs_find_cycle(vertex: str, parent: Optional[str], path: List[str]) -> Optional[List[str]]:
            """DFS that finds and returns a cycle."""
            visited.add(vertex)
            path.append(vertex)

            for neighbor in graph.get(vertex, set()):
                if neighbor not in visited:
                    result = dfs_find_cycle(neighbor, vertex, path[:])
                    if result:
                        return result
                elif neighbor != parent:
                    # Found cycle! Reconstruct it
                    cycle_start_idx = path.index(neighbor)
                    cycle = path[cycle_start_idx:] + [neighbor]
                    return cycle

            return None

        for vertex in graph:
            if vertex not in visited:
                cycle = dfs_find_cycle(vertex, None, [])
                if cycle:
```

```
return cycle
```

```
return None
```

```
# Find a cycle
```

```
cycle = find_cycle(artist_graph)
```

```
if cycle:
```

```
    print(f"\nFound cycle of length {len(cycle) - 1}:")
```

```
    for i, artist in enumerate(cycle):
```

```
        print(f"    {i}: {artist}")
```

```
else:
```

```
    print("\nNo cycles found (graph is a tree or forest)")
```

DFS applications

Real-world uses:

- **Cycle detection:** Detect circular dependencies in build systems
- **Connected components:** Find clusters in social networks
- **Topological sort:** Order tasks with dependencies
- **Maze solving:** Find path through maze (backtracking)
- **Puzzle solving:** Sudoku, N-Queens (backtracking)

Spotify applications:

- Find artist communities (connected components)
- Detect circular playlist references
- Explore genre neighborhoods deeply
- Recommendation via deep exploration

Complexity:

- Time: $O(V + E)$ — visit each vertex and edge once
- Space: $O(V)$ for visited set, $O(\text{depth})$ for stack/recursion

Exercise 2: Find music genre clusters

Task: Find connected components representing genre clusters.

Requirements:

1. Build a track similarity graph (tracks by same artist are connected)
2. Use DFS to find all connected components
3. For each component:
 - Count number of tracks
 - Find most common artist
 - Print component size and representative artist
4. Find the largest component (main genre cluster)

Hint: Use `find_connected_components` as a starting point.

Commit required: Commit your solution before class ends.

```
In [ ]: # YOUR CODE HERE
        # Find genre clusters using connected components

def build_track_graph(tracks):
    """Build graph where tracks by same artist are connected."""
    # TODO: Group tracks by artist
    # TODO: Create edges between tracks in same group
```

pass

```
def find_genre_clusters(track_graph, tracks):  
    """Find connected components representing genre clusters."""  
    # TODO: Use DFS to find components  
    # TODO: Analyze each component  
    pass  
  
    # Build graph and find clusters  
    # track_graph = build_track_graph(tracks)  
    # clusters = find_genre_clusters(track_graph, tracks)
```

Complexity summary

Time complexity:

- BFS: $O(V + E)$
- DFS: $O(V + E)$
- Shortest path (BFS): $O(V + E)$
- Connected components: $O(V + E)$
- Cycle detection: $O(V + E)$

Space complexity:

- BFS: $O(V)$ for queue and visited set
- DFS iterative: $O(V)$ for stack and visited set
- DFS recursive: $O(V)$ for visited, $O(\text{depth})$ for call stack

Both BFS and DFS visit each vertex and edge exactly once!

Wrap-up: BFS/DFS for routing and reachability

You've learned:

-  BFS uses queue (deque) for level-by-level exploration
-  BFS finds shortest paths in unweighted graphs
-  DFS uses stack (or recursion) for deep exploration
-  DFS detects cycles and finds connected components
-  Both have $O(V + E)$ time complexity
-  Choose BFS for shortest paths, DFS for cycles/components
-  Applied to Spotify: collaboration chains, artist clusters

Next lecture: Shortest paths in practice (Dijkstra, weighted graphs)

Don't forget: Commit both exercises before you leave!

Complexity checkpoints

Question 1: Why does BFS find shortest paths but DFS doesn't?

- A) BFS is faster
- B) BFS explores in order of increasing distance
- C) DFS doesn't visit all vertices
- D) BFS uses less memory

Question 2: What's the time complexity of finding all connected components?

- A) $O(V)$
- B) $O(E)$
- C) $O(V + E)$
- D) $O(V \times E)$

Question 3: Which data structure does BFS use?

- A) Stack
- B) Queue
- C) Heap
- D) Set

Answers: B (increasing distance), C ($O(V+E)$), B (queue)

